MainFile_pandas

October 26, 2026

1 Learning Pandas -~~~

i) we can change the index to make it more readable - whether its a tuple, list, a np.array etc.

[34]:
```
print(pd.Series(line, index=[1,2,3,3]))
print('')
names1= ['Stage1 |','Stage2 |','Stage3 |','Mega   |']
nametest =pd.Series(line,index=names1)
nametest
```

```
1       Bulbasaur
2         Ivysaur
3        Venusaur
3    MegaVenusaur
dtype: object
```

[34]:
```
Stage1 |       Bulbasaur
Stage2 |         Ivysaur
Stage3 |        Venusaur
Mega   |    MegaVenusaur
dtype: object
```

ii) we can print a specific value, i.e. when a label corresponds to a value using the series.loc[] function:

[35]:
```
print(nametest.loc["Stage3 |"],nametest.loc["Stage1 |"])
```

```
Venusaur Bulbasaur
```

iii) replacing a specific value using location:

[36]:
```
nametest.loc["Mega   |"] = 'GmaxVenusaur'
print(series)
print(nametest)
```

```
0       Bulbasaur
1         Ivysaur
2        Venusaur
3    MegaVenusaur
dtype: object
Stage1 |       Bulbasaur
Stage2 |         Ivysaur
Stage3 |        Venusaur
Mega   |    GmaxVenusaur
dtype: object
```

iv) this can be done via index with iloc (index location)

[37]:
```
print(nametest.iloc[3])
```

```
GmaxVenusaur
```

1.4 [2b] Filtering series

basic filtering.

i) start with a new data set. we will use this to do some filtering

[38]:
```
data = [66,88,200,200.5,100,180,10,300,202,400,800]
alphanumeric='abcdefghijklmnopqrstuvwxyz'

stringr = []

for i in alphanumeric:
    stringr+=i

numSeries =pd.Series(data,index=stringr[:len(data)])  #creates a sublist of the␣
 ↪alphabet for the index, only up to the length of the data
numSeries
```

[38]:
```
a     66.0
b     88.0
c    200.0
d    200.5
e    100.0
f    180.0
g     10.0
h    300.0
i    202.0
j    400.0
k    800.0
dtype: float64
```

ii) now create a filtered sub-series, for values greater than or equal to 200

[39]:
```
subnumero = numSeries[numSeries >=200]
subnumero
```

[39]:
```
c    200.0
d    200.5
h    300.0
i    202.0
j    400.0
k    800.0
dtype: float64
```

iii) now we will explore pandas with dictionaries :)

[40]:
```
calories = {"Day1":1750, "Day2":2100, "Day3":1700}
pizza = pd.Series(calories)
pizza #notice how the labels are automatically used as a column!!!
```

[40]:
```
Day1    1750
Day2    2100
Day3    1700
dtype: int64
```

[41]:
```
print(pizza.loc['Day3'])
pizza.iloc[2] = 3800 #updating again, this time with iloc
pizza
```

```
1700
```

[41]:
```
Day1    1750
Day2    2100
Day3    3800
dtype: int64
```

[42]:
```
print(pizza[pizza<2000]) #all the days i kept my diet :))
```

```
Day1    1750
dtype: int64
```

1.5 [3a] DataFrames in pandas

creating our first dataframe,

*i) dataframes, tables, excel sheets or tibbles as you know are just 2-dimensional grids of data (panel data)*let's try creating a list...

[3]:
```
print(list(range(6)), "testing range") #exploring range outside of lists; for␣
 ↪natdex ID

hoenn = {"NatDex_ID":list(range(252,261)),
        "Name":␣
 ↪["Treecko","Grovyle","Sceptile","Torchic","Combusken","Blaziken","Mudkip","Marshtomp","Swam
        "Type1":␣
 ↪["Grass","Grass","Grass","Fire","Fire","Fire","Water","Water","Water"],
        "Type2":␣
 ↪["Grass","Grass","Grass","Fire","Fighting","Fighting","Water","Ground","Ground"],
        "BST": [310,405,530,310,405,530,310,405,535]}

print("")
print('base python dictionary/"dataframe"',"""
""",hoenn) #this has just created a rather unusual dictionary. to convert this␣
 ↪into a dataframe (for useful analysis) see the next section:
```

```
[0, 1, 2, 3, 4, 5] testing range

base python dictionary/"dataframe"
 {'NatDex_ID': [252, 253, 254, 255, 256, 257, 258, 259, 260], 'Name':
['Treecko', 'Grovyle', 'Sceptile', 'Torchic', 'Combusken', 'Blaziken', 'Mudkip',
```

```
'Marshtomp', 'Swampert'], 'Type1': ['Grass', 'Grass', 'Grass', 'Fire', 'Fire',
'Fire', 'Water', 'Water', 'Water'], 'Type2': ['Grass', 'Grass', 'Grass', 'Fire',
'Fighting', 'Fighting', 'Water', 'Ground', 'Ground'], 'BST': [310, 405, 530,
310, 405, 530, 310, 405, 535]}
```

[4]:
```
   = pd.DataFrame(hoenn) #after the dictionary is made, this is actually very␣
 ↪simple!!!
print( )
print("")
   #compare printing to just pasting
```

```
   NatDex_ID       Name  Type1     Type2  BST
0        252    Treecko  Grass     Grass  310
1        253    Grovyle  Grass     Grass  405
2        254   Sceptile  Grass     Grass  530
3        255    Torchic   Fire      Fire  310
4        256  Combusken   Fire  Fighting  405
5        257   Blaziken   Fire  Fighting  530
6        258     Mudkip  Water     Water  310
7        259  Marshtomp  Water    Ground  405
8        260   Swampert  Water    Ground  535
```

[4]:

	NatDex_ID	Name	Type1	Type2	BST
0	252	Treecko	Grass	Grass	310
1	253	Grovyle	Grass	Grass	405
2	254	Sceptile	Grass	Grass	530
3	255	Torchic	Fire	Fire	310
4	256	Combusken	Fire	Fighting	405
5	257	Blaziken	Fire	Fighting	530
6	258	Mudkip	Water	Water	310
7	259	Marshtomp	Water	Ground	405
8	260	Swampert	Water	Ground	535

notice how there was still an index column created. Python will always have this, in this case may be easier to create a dF without the ID column

[5]:
```
hoenn2 = {"Name":␣
 ↪["Treecko","Grovyle","Sceptile","Torchic","Combusken","Blaziken","Mudkip","Marshtomp","Swam
       "Type1":␣
 ↪["Grass","Grass","Grass","Fire","Fire","Fire","Water","Water","Water"],
       "Type2": ["","","","","Fighting","Fighting","","Ground","Ground"],
       "BST": [310,405,530,310,405,530,310,405,535]}

starters = pd.DataFrame(hoenn2,index=list(range(252,261)))
starters
```

[5]:

	Name	Type1	Type2	BST
252	Treecko	Grass		310
253	Grovyle	Grass		405
254	Sceptile	Grass		530
255	Torchic	Fire		310
256	Combusken	Fire	Fighting	405
257	Blaziken	Fire	Fighting	530
258	Mudkip	Water		310
259	Marshtomp	Water	Ground	405
260	Swampert	Water	Ground	535

*slightly different; we moved NatDex IDs to the index column, and made single types have an empty column.*locating works the same, here is for both loc and iloc:

[6]:
```
print(starters.loc[252]+starters.loc[255]) #oops concatenation...
print(starters.iloc[8]) #standard - index 8 (9th value)
```

```
Name     TreeckoTorchic
Type1         GrassFire
Type2
BST                 620
dtype: object
Name     Swampert
Type1       Water
Type2      Ground
BST           535
Name: 260, dtype: object
```

1.6 [3b] Adding and Filtering Dataframes

similar to %>% mutate("NewCol" <- [1,2,3]); filter() etc.

i) adding a new column

[7]:
```
starters["EggGroup1"] =␣
 ↪['Monster','Monster','Monster','Field','Field','Field','Monster','Monster','Monster']
starters["EggGroup2"] =␣
 ↪['Dragon','Dragon','Dragon','','','','Water1','Water1','Water1']
starters["EggCycle"]  = [20] * 9
starters
```

[7]:

	Name	Type1	Type2	BST	EggGroup1	EggGroup2	EggCycle
252	Treecko	Grass		310	Monster	Dragon	20
253	Grovyle	Grass		405	Monster	Dragon	20
254	Sceptile	Grass		530	Monster	Dragon	20
255	Torchic	Fire		310	Field		20
256	Combusken	Fire	Fighting	405	Field		20
257	Blaziken	Fire	Fighting	530	Field		20
258	Mudkip	Water		310	Monster	Water1	20

259	Marshtomp	Water	Ground	405	Monster	Water1	20
260	Swampert	Water	Ground	535	Monster	Water1	20

ii) for adding new rows, the best way to do this is to create a new dataframe and concatenate it

[8]:
```
legends = pd.DataFrame([{"Name": "Kyogre", "Type1": "Water", "Type2": "", "BST":
 ↪ 670,"EggGroup1": "NoEggsDiscovered", "EggGroup2": "", "EggCycle": "N/A"},
                        {"Name": "PrimalKyogre", "Type1": "Water", "Type2": "",␣
 ↪"BST": 770,"EggGroup1": "NoEggsDiscovered", "EggGroup2": "", "EggCycle": "N/
 ↪A"},
                        {"Name": "Groudon", "Type1": "Ground", "Type2": "",␣
 ↪"BST": 670,"EggGroup1": "NoEggsDiscovered", "EggGroup2": "", "EggCycle": "N/
 ↪A"},
                        {"Name": "PrimalGroudon", "Type1": "Ground", "Type2":␣
 ↪"Fire", "BST": 770,"EggGroup1": "NoEggsDiscovered", "EggGroup2": "",␣
 ↪"EggCycle": "N/A"},
                        {"Name": "Rayquaza", "Type1": "Dragon", "Type2":␣
 ↪"Flying", "BST": 680,"EggGroup1": "NoEggsDiscovered", "EggGroup2": "",␣
 ↪"EggCycle": "N/A"},
                        {"Name": "RayquazaMega", "Type1": "Dragon", "Type2":␣
 ↪"Flying", "BST": 780,"EggGroup1": "NoEggsDiscovered", "EggGroup2": "",␣
 ↪"EggCycle": "N/A"}],
                       index=[382,382,383,383,384,384])

legends
```

[8]:

	Name	Type1	Type2	BST	EggGroup1	EggGroup2	EggCycle
382	Kyogre	Water		670	NoEggsDiscovered		N/A
382	PrimalKyogre	Water		770	NoEggsDiscovered		N/A
383	Groudon	Ground		670	NoEggsDiscovered		N/A
383	PrimalGroudon	Ground	Fire	770	NoEggsDiscovered		N/A
384	Rayquaza	Dragon	Flying	680	NoEggsDiscovered		N/A
384	RayquazaMega	Dragon	Flying	780	NoEggsDiscovered		N/A

now, lets concatenate them (replacing the append() function)

[9]:
```
bookends3 = pd.concat([starters,legends])
bookends3
```

[9]:

	Name	Type1	Type2	BST	EggGroup1	EggGroup2	EggCycle
252	Treecko	Grass		310	Monster	Dragon	20
253	Grovyle	Grass		405	Monster	Dragon	20
254	Sceptile	Grass		530	Monster	Dragon	20
255	Torchic	Fire		310	Field		20
256	Combusken	Fire	Fighting	405	Field		20
257	Blaziken	Fire	Fighting	530	Field		20
258	Mudkip	Water		310	Monster	Water1	20

```
259      Marshtomp   Water    Ground  405           Monster    Water1       20
260       Swampert   Water    Ground  535           Monster    Water1       20
382         Kyogre   Water            670  NoEggsDiscovered                N/A
382  PrimalKyogre    Water            770  NoEggsDiscovered                N/A
383        Groudon  Ground            670  NoEggsDiscovered                N/A
383  PrimalGroudon  Ground      Fire  770  NoEggsDiscovered                N/A
384       Rayquaza  Dragon    Flying  680  NoEggsDiscovered                N/A
384   RayquazaMega  Dragon    Flying  780  NoEggsDiscovered                N/A
```

iii) using iloc and various filters to select a range:

[10]:
```
bookends3.iloc[2:4] #using iloc allows you to select a range
```

[10]:
```
         Name  Type1 Type2  BST EggGroup1 EggGroup2 EggCycle
254  Sceptile  Grass        530   Monster    Dragon       20
255   Torchic   Fire        310     Field                 20
```

[11]:
```
bookends3[(bookends3['Type1'] == 'Fire') | (bookends3['Type2']== 'Fire')] #use␣
↪the same symbols (&,|,not) as in R; but for pandas
```

[11]:
```
              Name   Type1     Type2  BST         EggGroup1 EggGroup2 EggCycle
255        Torchic    Fire           310             Field                 20
256      Combusken    Fire  Fighting  405             Field                 20
257       Blaziken    Fire  Fighting  530             Field                 20
383  PrimalGroudon  Ground      Fire  770  NoEggsDiscovered                N/A
```

[12]:
```
bookends3[bookends3['BST'] > 500]
```

[12]:
```
              Name   Type1     Type2  BST         EggGroup1 EggGroup2 EggCycle
254       Sceptile   Grass           530           Monster    Dragon       20
257       Blaziken    Fire  Fighting  530             Field                 20
260       Swampert   Water    Ground  535           Monster    Water1       20
382         Kyogre   Water            670  NoEggsDiscovered                N/A
382  PrimalKyogre    Water            770  NoEggsDiscovered                N/A
383        Groudon  Ground            670  NoEggsDiscovered                N/A
383  PrimalGroudon  Ground      Fire  770  NoEggsDiscovered                N/A
384       Rayquaza  Dragon    Flying  680  NoEggsDiscovered                N/A
384   RayquazaMega  Dragon    Flying  780  NoEggsDiscovered                N/A
```

1.7 [4a] Importing Data

similar to R, we use read_delim() | read_csv() | read_excel() *(without having to import libraries readr/readxl lol)*

i) constructing our own CSV file:

No,Name,Type1,Type2,Height,Weight,Legendary 1,Bulbasaur,Grass,Poison,0.7,6.9,0 2,Ivysaur,Grass,Poison,1,13,0 3,Venusaur,Grass,Poison,2,100,0..... *you get the idea.. we'll call it Red&Blue.csv and read it in below*

[13]:
```
gen1 = pd.read_csv("Red&Blue.csv")
gen1
```

[13]:

	DexNo	Name	Type1	Type2	Height	Weight	Legendary	Mythical
0	1	Bulbasaur	Grass	Poison	0.7	6.9	0	0
1	2	Ivysaur	Grass	Poison	1.0	13.0	0	0
2	3	Venusaur	Grass	Poison	2.0	100.0	0	0
3	4	Charmander	Fire	NaN	0.6	8.5	0	0
4	5	Charmeleon	Fire	NaN	1.1	19.0	0	0
..	…	…	…	…	…	…	…	
146	147	Dratini	Dragon	NaN	1.8	3.3	0	0
147	148	Dragonair	Dragon	NaN	4.0	16.5	0	0
148	149	Dragonite	Dragon	Flying	2.2	210.0	0	0
149	150	Mewtwo	Psychic	NaN	2.0	122.0	1	0
150	151	Mew	Psychic	NaN	0.4	4.0	0	1

[151 rows x 8 columns]

[14]:
```
print(gen1.to_string()) #prints the whole thing.. be careful if file too large
↪ur pc may poop
```

	DexNo	Name	Type1	Type2	Height	Weight	Legendary	Mythical
0	1	Bulbasaur	Grass	Poison	0.7	6.9	0	0
1	2	Ivysaur	Grass	Poison	1.0	13.0	0	0
2	3	Venusaur	Grass	Poison	2.0	100.0	0	0
3	4	Charmander	Fire	NaN	0.6	8.5	0	0
4	5	Charmeleon	Fire	NaN	1.1	19.0	0	0
5	6	Charizard	Fire	Flying	1.7	90.5	0	0
6	7	Squirtle	Water	NaN	0.5	9.0	0	0
7	8	Wartortle	Water	NaN	1.0	22.5	0	0
8	9	Blastoise	Water	NaN	1.6	85.5	0	0
9	10	Caterpie	Bug	NaN	0.3	2.9	0	0
10	11	Metapod	Bug	NaN	0.7	9.9	0	0
11	12	Butterfree	Bug	Flying	1.1	32.0	0	0
12	13	Weedle	Bug	Poison	0.3	3.2	0	0
13	14	Kakuna	Bug	Poison	0.6	10.0	0	0
14	15	Beedrill	Bug	Poison	1.0	29.5	0	0
15	16	Pidgey	Normal	Flying	0.3	1.8	0	0
16	17	Pidgeotto	Normal	Flying	1.1	30.0	0	0
17	18	Pidgeot	Normal	Flying	1.5	39.5	0	0
18	19	Rattata	Normal	NaN	0.3	3.5	0	0
19	20	Raticate	Normal	NaN	0.7	18.5	0	0
20	21	Spearow	Normal	Flying	0.3	2.0	0	0
21	22	Fearow	Normal	Flying	1.2	38.0	0	0
22	23	Ekans	Poison	NaN	2.0	6.9	0	0
23	24	Arbok	Poison	NaN	3.5	65.0	0	0
24	25	Pikachu	Electric	NaN	0.4	6.0	0	0
25	26	Raichu	Electric	NaN	0.8	30.0	0	0

26	27	Sandshrew	Ground	NaN	0.6	12.0	0	0
27	28	Sandslash	Ground	NaN	1.0	29.5	0	0
28	29	Nidoran	Poison	NaN	0.4	7.0	0	0
29	30	Nidorina	Poison	NaN	0.8	20.0	0	0
30	31	Nidoqueen	Poison	Ground	1.3	60.0	0	0
31	32	Nidoran	Poison	NaN	0.5	9.0	0	0
32	33	Nidorino	Poison	NaN	0.9	19.5	0	0
33	34	Nidoking	Poison	Ground	1.4	62.0	0	0
34	35	Clefairy	Fairy	NaN	0.6	7.5	0	0
35	36	Clefable	Fairy	NaN	1.3	40.0	0	0
36	37	Vulpix	Fire	NaN	0.6	9.9	0	0
37	38	Ninetales	Fire	NaN	1.1	19.9	0	0
38	39	Jigglypuff	Normal	Fairy	0.5	5.5	0	0
39	40	Wigglytuff	Normal	Fairy	1.0	12.0	0	0
40	41	Zubat	Poison	Flying	0.8	7.5	0	0
41	42	Golbat	Poison	Flying	1.6	55.0	0	0
42	43	Oddish	Grass	Poison	0.5	5.4	0	0
43	44	Gloom	Grass	Poison	0.8	8.6	0	0
44	45	Vileplume	Grass	Poison	1.2	18.6	0	0
45	46	Paras	Bug	Grass	0.3	5.4	0	0
46	47	Parasect	Bug	Grass	1.0	29.5	0	0
47	48	Venonat	Bug	Poison	1.0	30.0	0	0
48	49	Venomoth	Bug	Poison	1.5	12.5	0	0
49	50	Diglett	Ground	NaN	0.2	0.8	0	0
50	51	Dugtrio	Ground	NaN	0.7	33.3	0	0
51	52	Meowth	Normal	NaN	0.4	4.2	0	0
52	53	Persian	Normal	NaN	1.0	32.0	0	0
53	54	Psyduck	Water	NaN	0.8	19.6	0	0
54	55	Golduck	Water	NaN	1.7	76.6	0	0
55	56	Mankey	Fighting	NaN	0.5	28.0	0	0
56	57	Primeape	Fighting	NaN	1.0	32.0	0	0
57	58	Growlithe	Fire	NaN	0.7	19.0	0	0
58	59	Arcanine	Fire	NaN	1.9	155.0	0	0
59	60	Poliwag	Water	NaN	0.6	12.4	0	0
60	61	Poliwhirl	Water	NaN	1.0	20.0	0	0
61	62	Poliwrath	Water	Fighting	1.3	54.0	0	0
62	63	Abra	Psychic	NaN	0.9	19.5	0	0
63	64	Kadabra	Psychic	NaN	1.3	56.5	0	0
64	65	Alakazam	Psychic	NaN	1.5	48.0	0	0
65	66	Machop	Fighting	NaN	0.8	19.5	0	0
66	67	Machoke	Fighting	NaN	1.5	70.5	0	0
67	68	Machamp	Fighting	NaN	1.6	130.0	0	0
68	69	Bellsprout	Grass	Poison	0.7	4.0	0	0
69	70	Weepinbell	Grass	Poison	1.0	6.4	0	0
70	71	Victreebel	Grass	Poison	1.7	15.5	0	0
71	72	Tentacool	Water	Poison	0.9	45.5	0	0
72	73	Tentacruel	Water	Poison	1.6	55.0	0	0
73	74	Geodude	Rock	Ground	0.4	20.0	0	0

74	75	Graveler	Rock	Ground	1.0	105.0	0	0
75	76	Golem	Rock	Ground	1.4	300.0	0	0
76	77	Ponyta	Fire	NaN	1.0	30.0	0	0
77	78	Rapidash	Fire	NaN	1.7	95.0	0	0
78	79	Slowpoke	Water	Psychic	1.2	36.0	0	0
79	80	Slowbro	Water	Psychic	1.6	78.5	0	0
80	81	Magnemite	Electric	Steel	0.3	6.0	0	0
81	82	Magneton	Electric	Steel	1.0	60.0	0	0
82	83	Farfetch'd	Normal	Flying	0.8	15.0	0	0
83	84	Doduo	Normal	Flying	1.4	39.2	0	0
84	85	Dodrio	Normal	Flying	1.8	85.2	0	0
85	86	Seel	Water	NaN	1.1	90.0	0	0
86	87	Dewgong	Water	Ice	1.7	120.0	0	0
87	88	Grimer	Poison	NaN	0.9	30.0	0	0
88	89	Muk	Poison	NaN	1.2	30.0	0	0
89	90	Shellder	Water	NaN	0.3	4.0	0	0
90	91	Cloyster	Water	Ice	1.5	132.5	0	0
91	92	Gastly	Ghost	Poison	1.3	0.1	0	0
92	93	Haunter	Ghost	Poison	1.6	0.1	0	0
93	94	Gengar	Ghost	Poison	1.5	40.5	0	0
94	95	Onix	Rock	Ground	8.8	210.0	0	0
95	96	Drowzee	Psychic	NaN	1.0	32.4	0	0
96	97	Hypno	Psychic	NaN	1.6	75.6	0	0
97	98	Krabby	Water	NaN	0.4	6.5	0	0
98	99	Kingler	Water	NaN	1.3	60.0	0	0
99	100	Voltorb	Electric	NaN	0.5	10.4	0	0
100	101	Electrode	Electric	NaN	1.2	66.6	0	0
101	102	Exeggcute	Grass	Psychic	0.4	2.5	0	0
102	103	Exeggutor	Grass	Psychic	2.0	120.0	0	0
103	104	Cubone	Ground	NaN	0.4	6.5	0	0
104	105	Marowak	Ground	NaN	1.0	45.0	0	0
105	106	Hitmonlee	Fighting	NaN	1.5	49.8	0	0
106	107	Hitmonchan	Fighting	NaN	1.4	50.2	0	0
107	108	Lickitung	Normal	NaN	1.2	65.5	0	0
108	109	Koffing	Poison	NaN	0.6	1.0	0	0
109	110	Weezing	Poison	NaN	1.2	9.5	0	0
110	111	Rhyhorn	Ground	Rock	1.0	115.0	0	0
111	112	Rhydon	Ground	Rock	1.9	120.0	0	0
112	113	Chansey	Normal	NaN	1.1	34.6	0	0
113	114	Tangela	Grass	NaN	1.0	35.0	0	0
114	115	Kangaskhan	Normal	NaN	2.2	80.0	0	0
115	116	Horsea	Water	NaN	0.4	8.0	0	0
116	117	Seadra	Water	NaN	1.2	25.0	0	0
117	118	Goldeen	Water	NaN	0.6	15.0	0	0
118	119	Seaking	Water	NaN	1.3	39.0	0	0
119	120	Staryu	Water	NaN	0.8	34.5	0	0
120	121	Starmie	Water	Psychic	1.1	80.0	0	0
121	122	Mr. Mime	Psychic	Fairy	1.3	54.5	0	0

122	123	Scyther	Bug	Flying	1.5	56.0	0	0
123	124	Jynx	Ice	Psychic	1.4	40.6	0	0
124	125	Electabuzz	Electric	NaN	1.1	30.0	0	0
125	126	Magmar	Fire	NaN	1.3	44.5	0	0
126	127	Pinsir	Bug	NaN	1.5	55.0	0	0
127	128	Tauros	Normal	NaN	1.4	88.4	0	0
128	129	Magikarp	Water	NaN	0.9	10.0	0	0
129	130	Gyarados	Water	Flying	6.5	235.0	0	0
130	131	Lapras	Water	Ice	2.5	220.0	0	0
131	132	Ditto	Normal	NaN	0.3	4.0	0	0
132	133	Eevee	Normal	NaN	0.3	6.5	0	0
133	134	Vaporeon	Water	NaN	1.0	29.0	0	0
134	135	Jolteon	Electric	NaN	0.8	24.5	0	0
135	136	Flareon	Fire	NaN	0.9	25.0	0	0
136	137	Porygon	Normal	NaN	0.8	36.5	0	0
137	138	Omanyte	Rock	Water	0.4	7.5	0	0
138	139	Omastar	Rock	Water	1.0	35.0	0	0
139	140	Kabuto	Rock	Water	0.5	11.5	0	0
140	141	Kabutops	Rock	Water	1.3	40.5	0	0
141	142	Aerodactyl	Rock	Flying	1.8	59.0	0	0
142	143	Snorlax	Normal	NaN	2.1	460.0	0	0
143	144	Articuno	Ice	Flying	1.7	55.4	1	0
144	145	Zapdos	Electric	Flying	1.6	52.6	1	0
145	146	Moltres	Fire	Flying	2.0	60.0	1	0
146	147	Dratini	Dragon	NaN	1.8	3.3	0	0
147	148	Dragonair	Dragon	NaN	4.0	16.5	0	0
148	149	Dragonite	Dragon	Flying	2.2	210.0	0	0
149	150	Mewtwo	Psychic	NaN	2.0	122.0	1	0
150	151	Mew	Psychic	NaN	0.4	4.0	0	1

ii) what is a JSON file?a JSON (JavaScript Object Notation) file is another way to store data. .json files are a hierarchical, typed format - hence ideal for structured objects, arrays, and APIs. CSVs are smaller and easier for Excel, whereas JSON supports nested data, explicit types, and is standard for web applications.

we won't do it here, but you would do the exact same thing (this time using pd.read_json(Red&Blue.json)), and the output would be exactly the same.

1.8 [5] Selecting columns and row (advanced filtering)

exact same as select() from dplyr, except slightly cleaner syntax *(BONUS: pokemon challenge inside!!)*

i) starting with gen1; we want to only select their name and weight

[15]:
```
gen1
```

[15]:

	DexNo	Name	Type1	Type2	Height	Weight	Legendary	Mythical
0	1	Bulbasaur	Grass	Poison	0.7	6.9	0	0

```
1        2     Ivysaur    Grass  Poison     1.0    13.0          0          0
2        3    Venusaur    Grass  Poison     2.0   100.0          0          0
3        4  Charmander     Fire     NaN     0.6     8.5          0          0
4        5  Charmeleon     Fire     NaN     1.1    19.0          0          0
..     …         …      …     …      …     …         …        …
146    147     Dratini   Dragon     NaN     1.8     3.3          0          0
147    148   Dragonair   Dragon     NaN     4.0    16.5          0          0
148    149   Dragonite   Dragon  Flying     2.2   210.0          0          0
149    150      Mewtwo  Psychic     NaN     2.0   122.0          1          0
150    151         Mew  Psychic     NaN     0.4     4.0          0          1

[151 rows x 8 columns]
```

[16]:
```
gen1['Name'] #this has reverted to a series as you can see
```

[16]:
```
0       Bulbasaur
1         Ivysaur
2        Venusaur
3      Charmander
4      Charmeleon
          …
146       Dratini
147     Dragonair
148     Dragonite
149        Mewtwo
150           Mew
Name: Name, Length: 151, dtype: object
```

ii) great, but this syntax only works for one column.. how to do multiple/custom?we do this by nested brackets (see below)

[17]:
```
gen1[['Name','Height','Weight']]
```

[17]:
```
           Name  Height  Weight
0     Bulbasaur     0.7     6.9
1       Ivysaur     1.0    13.0
2      Venusaur     2.0   100.0
3    Charmander     0.6     8.5
4    Charmeleon     1.1    19.0
..          …     …      …
146     Dratini     1.8     3.3
147   Dragonair     4.0    16.5
148   Dragonite     2.2   210.0
149      Mewtwo     2.0   122.0
150         Mew     0.4     4.0

[151 rows x 3 columns]
```

iii) this is all great, but searching by national dex number is tough.. how can we

change this? when calling the dataframe, we can change the index column to correspond to names *(and hence when we use df.loc[] we can type a pokemon's name)*

[18]:
```
    = pd.read_csv('Red&Blue.csv',index_col='Name') #
```

[18]:

Name	DexNo	Type1	Type2	Height	Weight	Legendary	Mythical
Bulbasaur	1	Grass	Poison	0.7	6.9	0	0
Ivysaur	2	Grass	Poison	1.0	13.0	0	0
Venusaur	3	Grass	Poison	2.0	100.0	0	0
Charmander	4	Fire	NaN	0.6	8.5	0	0
Charmeleon	5	Fire	NaN	1.1	19.0	0	0
…	…	…	…	…	…	…	…
Dratini	147	Dragon	NaN	1.8	3.3	0	0
Dragonair	148	Dragon	NaN	4.0	16.5	0	0
Dragonite	149	Dragon	Flying	2.2	210.0	0	0
Mewtwo	150	Psychic	NaN	2.0	122.0	1	0
Mew	151	Psychic	NaN	0.4	4.0	0	1

[151 rows x 7 columns]

[53]:
```
print(    .loc['Mew'])
```

```
DexNo            151
Type1        Psychic
Type2            NaN
Height           0.4
Weight           4.0
Legendary          0
Mythical           1
Name: Mew, dtype: object
```

[54]:
```
#we can also pass in a second argument, selecting the columns we want to display
print(    .loc['Lapras',['Type1','Type2']])
```

```
Type1    Water
Type2      Ice
Name: Lapras, dtype: object
```

[55]:
```
#we can also do a slice; selecting rows from a certain range (or filtering as␣
 ↪before)
print(    .loc['Articuno':'Moltres',['Type1','Type2']])
```

Name	Type1	Type2
Articuno	Ice	Flying
Zapdos	Electric	Flying
Moltres	Fire	Flying

[56]:
```
#finally we can do the same, using index locators
print(    .iloc[2:9:3,[0,1,2,6]])
```

```
           DexNo  Type1   Type2  Mythical
Name
Venusaur       3  Grass  Poison         0
Charizard      6   Fire  Flying         0
Blastoise      9  Water     NaN         0
```

[57]:
```
#another example:
   =     .iloc[[150,149,143,144,145],[0,1,2,5,6]]
```

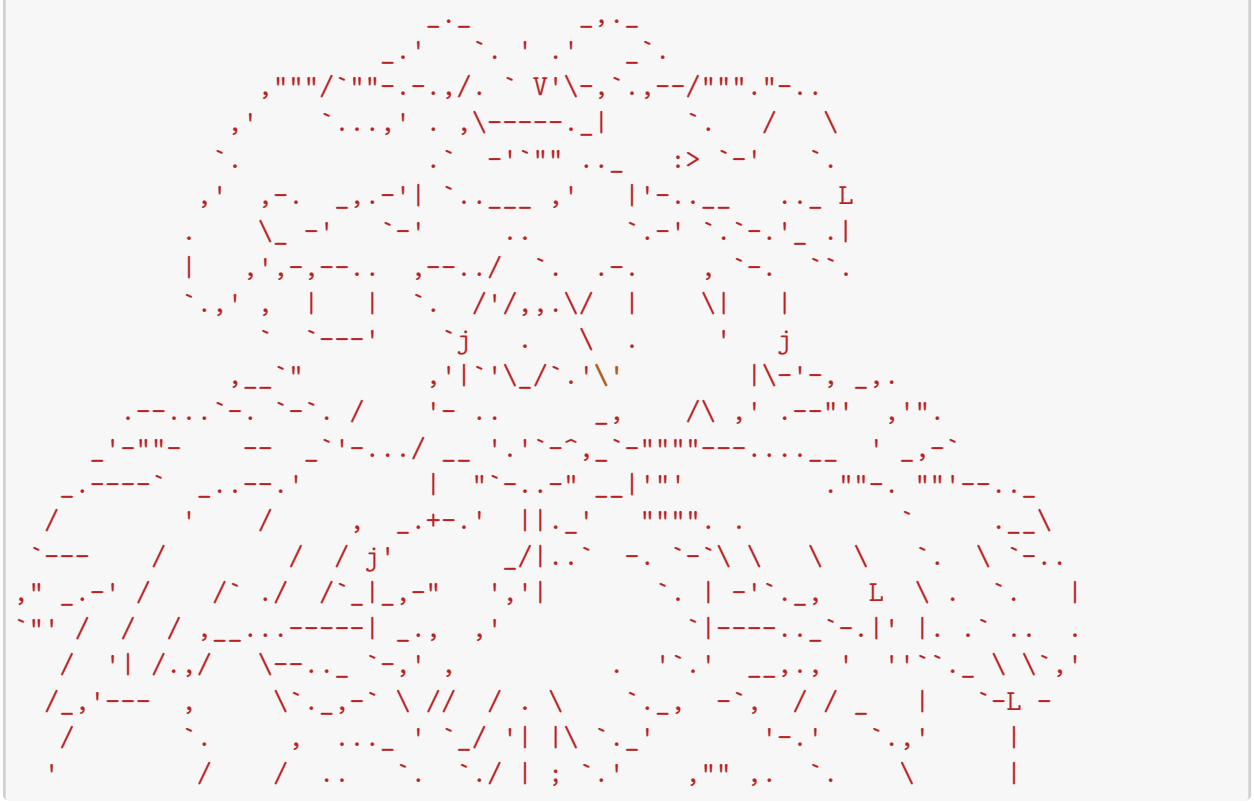
[57]:

Name	DexNo	Type1	Type2	Legendary	Mythical
Mew	151	Psychic	NaN	0	1
Mewtwo	150	Psychic	NaN	1	0
Articuno	144	Ice	Flying	1	0
Zapdos	145	Electric	Flying	1	0
Moltres	146	Fire	Flying	1	0

1.8.1 (iv) Mini project: Pokedex

does a pokedex search (plus surrounding strings); then a more general case

[19]:
```
welcome='''
```



```
   `.     ,'   ,'   | |\  |       "        |  ,'\ |   \    `    ,L
   /|`.  /    '     | `-| '                 /`-' |    L    `._/  \
  / | .`|    |  .    `._.'                  `.__,'   .  |     |  (`
 '-""-'_|    `. `.__,._____      .    _,        ____ ,-  j     ".-'"'
        \      `-.  \/.    `"--..._    _,.---'""\/  "_,.'     /-'
         )        `-._ '-.         `--"      _.-'.-""         `.
        ./            `,. `".._________..."".-"`.           _j
       /_\.__,"".   ,.'  "`-...________.---"     .".   ,.  / \
             \_/"""-'                            `-'--(_,`"`-`
                                ,'\
    _.----.        ____        ,'  _\   ___    ___     ____
_,-'       `.     |    |  /`.   \,-'    |   \  /   |   |    \  |`.
\      __    \    '-.  | /   `.  ___    |    \/    |   '-.   \ |  |
 \.    \ \   |  __  |  |/    ,','_   `.  |          | __  |    \|  |
   \    \/   /,' _`.|      ,' / / / /   |          ,' _`.|     |  |
    \     ,-'/  /   \    ,'   | \/ / ,`.|         /  /   \  |     |
     \    \ |   \_/  |   `-.  \    `'  /|  |    ||   \_/  | |\    |
      \    \ \      /       `-.`.___,-' |  |\  /| \      /  | |   |
       \    \ `.__,'|  |`-._    `|      |__| \/ |  `.__,'|  | |   |
        \_.-'       |__|    `-._ |              '-.|     '-.| |   |
                              `'                            '-._|
'''
print(welcome)
print("""

""")

pokemon=input("WELCOME to the world of Pokémon! Please type in a pokémon's name␣
 ↪to search their entry in the Kanto Pokédex: ")
print("""

""")
x=False
time.sleep(0.1)
try:
    print(    .loc[pokemon.title()])
    pokedex=    .loc[pokemon.title(), 'DexNo']
    x=True
except KeyError:
    print(f"{pokemon} not found.")
time.sleep(0.2)
if x:
    print()
    print()
    print(    .iloc[pokedex-3:pokedex+2])
```

-·- -’·-



WELCOME to the world of Pokémon! Please type in a pokémon's name to search their

```
entry in the Kanto Pokédex:  pikachu

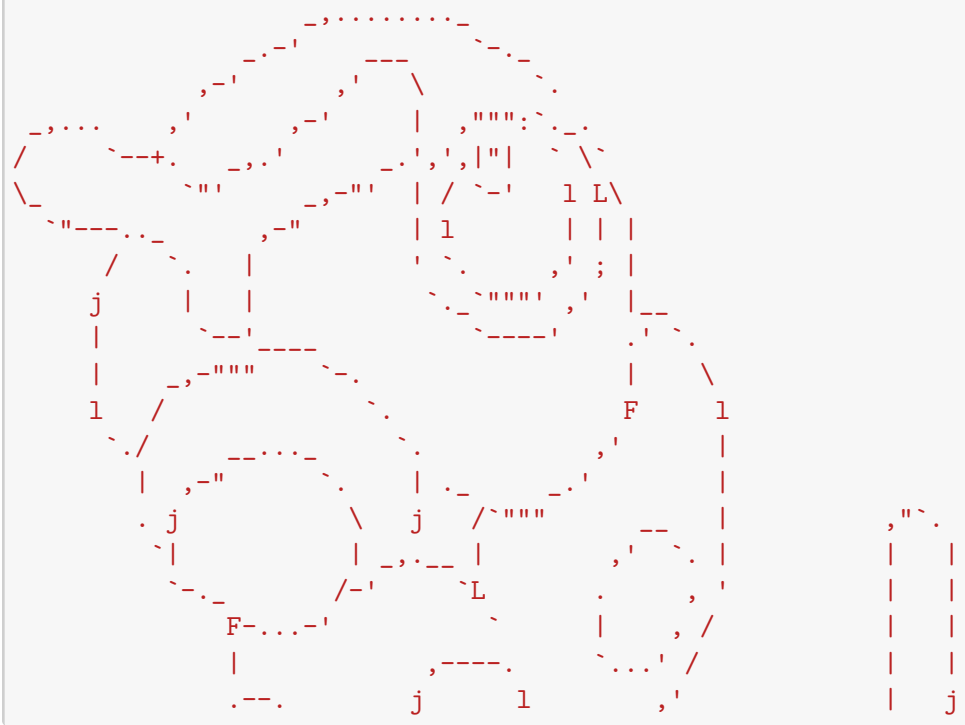


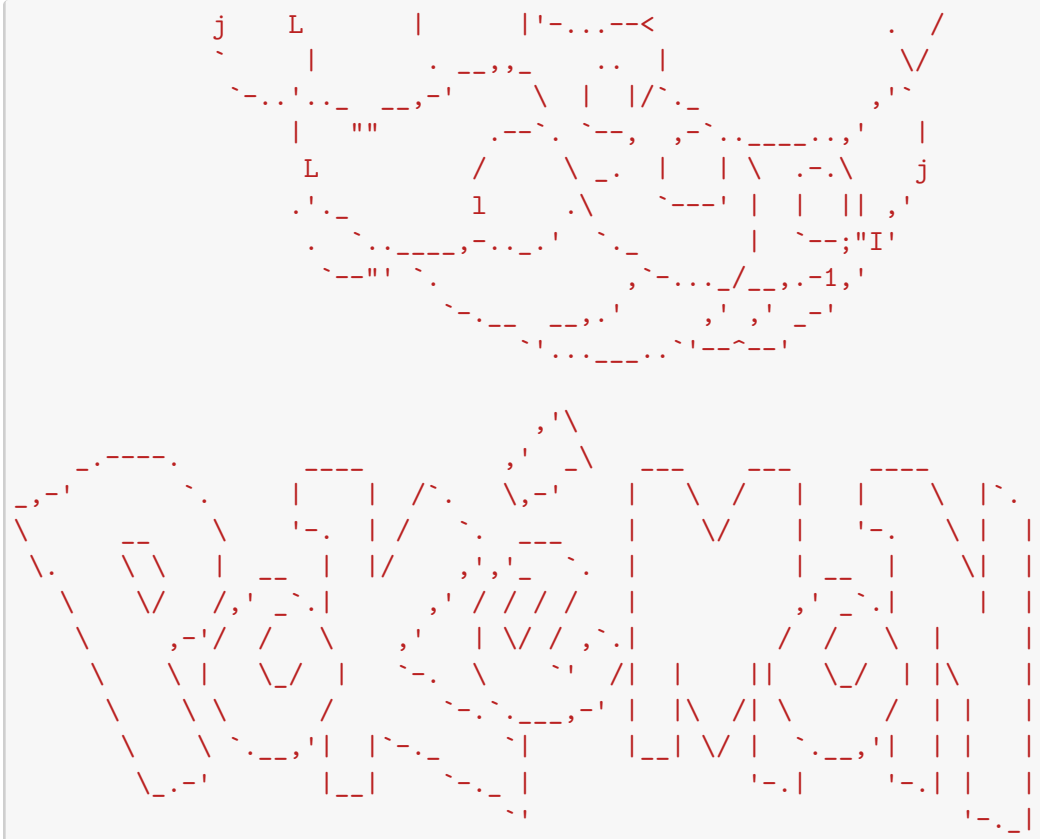
DexNo              25
Type1        Electric
Type2             NaN
Height            0.4
Weight            6.0
Legendary           0
Mythical            0
Name: Pikachu, dtype: object
```

```
           DexNo     Type1 Type2  Height  Weight  Legendary  Mythical
Name
Ekans         23    Poison   NaN     2.0     6.9          0         0
Arbok         24    Poison   NaN     3.5    65.0          0         0
Pikachu       25  Electric   NaN     0.4     6.0          0         0
Raichu        26  Electric   NaN     0.8    30.0          0         0
Sandshrew     27    Ground   NaN     0.6    12.0          0         0
```

using filters to try and do a predictive search (e.g. "nid" gives nidoking, nidoranF, nidoranM, nidoqueen, nidorino, nidorina)

[21]:
```
##########################################################################################
caterpie='''
```





```
'''
print(caterpie)
print("""

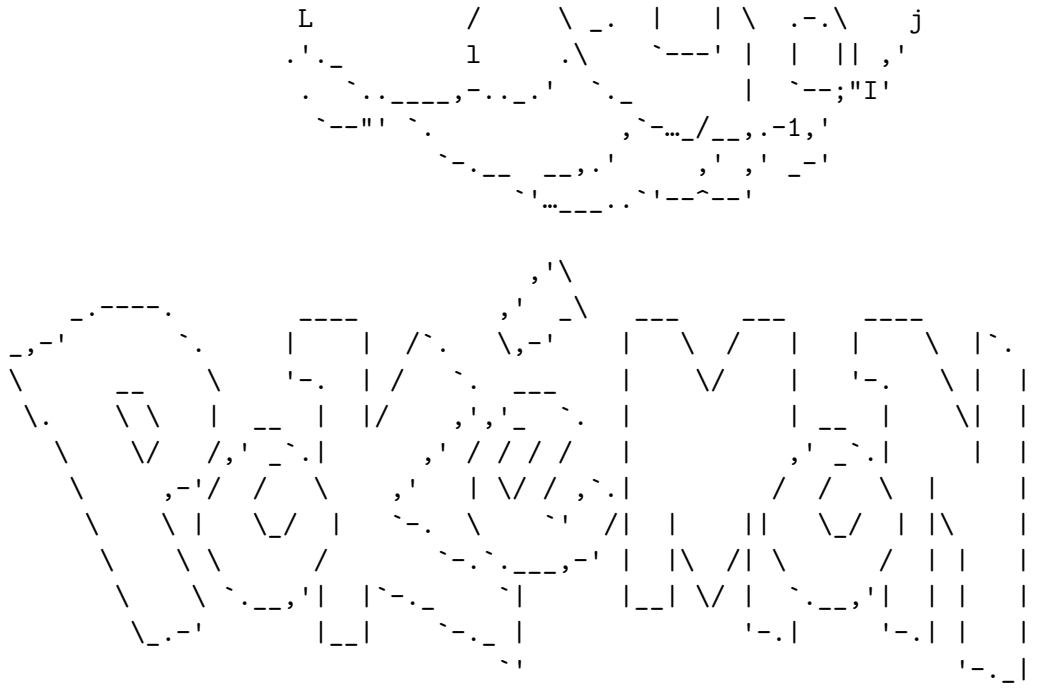
""")

zukan=input("WELCOME to the world of Pokémon! Please use the pokédex search to␣
 ↪find your favourite pokemon: ")
print("""
""")
x2=False
time.sleep(0.1)
try:
    filteredzukan =    [    .index.str.startswith(zukan.title()) == True]
    #here there's several parts. using .startswidth isntead of 'X' in 'Y' for␣
 ↪more precise searching;
    #and also have to add a .index since name is the index column, and str to␣
 ↪convert index/series to 'string'
    if len(filteredzukan.index) >1: #only takes the names column, we only need␣
 ↪1 column to test for length (doesnt matter)
        print(filteredzukan.to_string())
        time.sleep(0.2)
        print()
```

```
        kensaku=input(f'These are the search results for the search term␣
 ↪"{zukan}". Please type in the specific pokémon you would like to see␣
 ↪information on: ')
    elif len(filteredzukan.index) ==1:
        kensaku=zukan
    try:
        print()
        time.sleep(0.2)
        print(    .loc[kensaku.title()])
        pokedex2=    .loc[kensaku.title(), 'DexNo']
        x2=True
    except KeyError:
        print(f"{kensaku} not found.")


except KeyError:
    print(f"{zukan} not found.")
time.sleep(0.2)
if x2:
    print()
    print()
    print(    .iloc[pokedex2-3:pokedex2+2])
```





WELCOME to the world of Pokémon! Please use the pokédex search to find your favourite pokemon: s

Name	DexNo	Type1	Type2	Height	Weight	Legendary	Mythical
Squirtle	7	Water	NaN	0.5	9.0	0	0
Spearow	21	Normal	Flying	0.3	2.0	0	0
Sandshrew	27	Ground	NaN	0.6	12.0	0	0
Sandslash	28	Ground	NaN	1.0	29.5	0	0
Slowpoke	79	Water	Psychic	1.2	36.0	0	0
Slowbro	80	Water	Psychic	1.6	78.5	0	0
Seel	86	Water	NaN	1.1	90.0	0	0
Shellder	90	Water	NaN	0.3	4.0	0	0
Seadra	117	Water	NaN	1.2	25.0	0	0
Seaking	119	Water	NaN	1.3	39.0	0	0
Staryu	120	Water	NaN	0.8	34.5	0	0
Starmie	121	Water	Psychic	1.1	80.0	0	0
Scyther	123	Bug	Flying	1.5	56.0	0	0
Snorlax	143	Normal	NaN	2.1	460.0	0	0

These are the search results for the search term "s". Please type in the

```
specific pokémon you would like to see information on:  starMIE


DexNo            121
Type1          Water
Type2        Psychic
Height           1.1
Weight          80.0
Legendary          0
Mythical           0
Name: Starmie, dtype: object


          DexNo    Type1    Type2  Height  Weight  Legendary  Mythical
Name
Seaking     119    Water      NaN     1.3    39.0          0         0
Staryu      120    Water      NaN     0.8    34.5          0         0
Starmie     121    Water  Psychic     1.1    80.0          0         0
Mr. Mime    122  Psychic    Fairy     1.3    54.5          0         0
Scyther     123      Bug   Flying     1.5    56.0          0         0
```

1.9 [6] Aggregation / Summary Data

basic mathematical operators (mean,sum,min/max)

i) basic summary data

[91]:
```
gen1_means = (gen1.mean(numeric_only = True))
print(gen1_means)
```

```
DexNo        76.000000
Height        1.194702
Weight       45.951656
Legendary     0.026490
Mythical      0.006623
dtype: float64
```

[92]:
```
gen1_sums = (gen1.sum(numeric_only = True))
print(gen1_sums)
```

```
DexNo        11476.0
Height         180.4
Weight        6938.7
Legendary        4.0
Mythical         1.0
dtype: float64
```

[94]:
```
gen1_min = (gen1.min(numeric_only = True))
print(gen1_min)
```

```
print()
gen1_max = (gen1.max(numeric_only = True))
print(gen1_max)
```

```
DexNo        1.0
Height       0.2
Weight       0.1
Legendary    0.0
Mythical     0.0
dtype: float64

DexNo        151.0
Height         8.8
Weight       460.0
Legendary      1.0
Mythical       1.0
dtype: float64
```

count is slightly different - number of values in each column

[95]:
```
gen1.count()
```

[95]:
```
DexNo        151
Name         151
Type1        151
Type2         67
Height       151
Weight       151
Legendary    151
Mythical     151
dtype: int64
```

ii) summary statistics

[96]:
```
gen1.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 151 entries, 0 to 150
Data columns (total 8 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   DexNo      151 non-null    int64
 1   Name       151 non-null    object
 2   Type1      151 non-null    object
 3   Type2      67 non-null     object
 4   Height     151 non-null    float64
 5   Weight     151 non-null    float64
 6   Legendary  151 non-null    int64
 7   Mythical   151 non-null    int64
```

```
dtypes: float64(2), int64(3), object(3)
memory usage: 9.6+ KB
```

[98]:
```
gen1[['Type1','Type2']].describe()
```

[98]:
```
        Type1   Type2
count     151      67
unique     15      11
top     Water  Poison
freq       28      19
```

[99]:
```
gen1.describe()
```

[99]:
```
             DexNo      Height      Weight   Legendary    Mythical
count  151.000000  151.000000  151.000000  151.000000  151.000000
mean    76.000000    1.194702   45.951656    0.026490    0.006623
std     43.734045    0.962621   59.447988    0.161122    0.081379
min      1.000000    0.200000    0.100000    0.000000    0.000000
25%     38.500000    0.700000    9.900000    0.000000    0.000000
50%     76.000000    1.000000   30.000000    0.000000    0.000000
75%    113.500000    1.500000   56.250000    0.000000    0.000000
max    151.000000    8.800000  460.000000    1.000000    1.000000
```

[97]:
```
gen1.describe(include='all')
```

[97]:
```
             DexNo       Name  Type1   Type2      Height      Weight  \
count   151.000000        151    151      67  151.000000  151.000000
unique         NaN        151     15      11         NaN         NaN
top            NaN  Bulbasaur  Water  Poison         NaN         NaN
freq           NaN          1     28      19         NaN         NaN
mean     76.000000        NaN    NaN     NaN    1.194702   45.951656
std      43.734045        NaN    NaN     NaN    0.962621   59.447988
min       1.000000        NaN    NaN     NaN    0.200000    0.100000
25%      38.500000        NaN    NaN     NaN    0.700000    9.900000
50%      76.000000        NaN    NaN     NaN    1.000000   30.000000
75%     113.500000        NaN    NaN     NaN    1.500000   56.250000
max     151.000000        NaN    NaN     NaN    8.800000  460.000000

         Legendary    Mythical
count   151.000000  151.000000
unique         NaN         NaN
top            NaN         NaN
freq           NaN         NaN
mean      0.026490    0.006623
std       0.161122    0.081379
min       0.000000    0.000000
25%       0.000000    0.000000
50%       0.000000    0.000000
```

```
75%        0.000000    0.000000
max        1.000000    1.000000
```

we will try now to do pivot tables (two ways); then do other things

[102]:
```
gen1types = gen1.groupby('Type1')
print(gen1types)
```

```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x000002C699B09F90>
```

[107]:
```
print(gen1types['Height'].mean())
print(gen1types['Height'].count())
```

```
Type1
Bug         0.900000
Dragon      2.666667
Electric    0.855556
Fairy       0.950000
Fighting    1.185714
Fire        1.216667
Ghost       1.466667
Grass       1.083333
Ground      0.850000
Ice         1.550000
Normal      0.986364
Poison      1.221429
Psychic     1.250000
Rock        1.844444
Water       1.300000
Name: Height, dtype: float64
Type1
Bug         12
Dragon       3
Electric     9
Fairy        2
Fighting     7
Fire        12
Ghost        3
Grass       12
Ground       8
Ice          2
Normal      22
Poison      14
Psychic      8
Rock         9
Water       28
Name: Height, dtype: int64
```

1.10 [7] Pivot Tables

done in both possible ways, an explicit functions or R-style

*pandas.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean', fill_value=None, margins=False, dropna=True, margins_name='All', observed=True, sort=True, **kwargs)*

[317]:
```
frequency = gen1.groupby('Type1')
frequency = frequency['Name'].count()
print(frequency)

frequency.info()
widetypes1 = pd.
  ↪pivot_table(gen1,values=['Height','Weight'],columns='Type1',aggfunc='mean')
widetypes1.loc['Index'] = list(range(1,16))
widetypes1.loc['Frequency'] = frequency

widetypes1.iloc[3] = widetypes1.iloc[3].astype(int)
```

```
Type1
Bug         12
Dragon       3
Electric     9
Fairy        2
Fighting     7
Fire        12
Ghost        3
Grass       12
Ground       8
Ice          2
Normal      22
Poison      14
Psychic      8
Rock         9
Water       28
Name: Name, dtype: int64
<class 'pandas.core.series.Series'>
Index: 15 entries, Bug to Water
Series name: Name
Non-Null Count  Dtype
--------------  -----
15 non-null     int64
dtypes: int64(1)
memory usage: 240.0+ bytes
```

[201]:
```
widetypes1
```

[201]:
```
Type1            Bug     Dragon   Electric  Fairy   Fighting       Fire  \
Height      0.900000   2.666667   0.855556   0.95   1.185714   1.216667
Weight     22.991667  76.600000  31.788889  23.75  54.285714  48.025000
Index       1.000000   2.000000   3.000000   4.00   5.000000   6.000000
Frequency  12.000000   3.000000   9.000000   2.00   7.000000  12.000000

Type1          Ghost      Grass   Ground    Ice     Normal     Poison  \
Height      1.466667   1.083333   0.8500   1.55   0.986364   1.221429
Weight     13.566667  27.991667  45.2625  48.00  50.086364  27.314286
Index       7.000000   8.000000   9.0000  10.00  11.000000  12.000000
Frequency   3.000000  12.000000   8.0000   2.00  22.000000  14.000000

Type1       Psychic       Rock      Water
Height       1.2500   1.844444   1.300000
Weight      51.5625  87.611111  57.967857
Index       13.0000  14.000000  15.000000
Frequency    8.0000   9.000000  28.000000
```

[319]:
```
widetypes1.insert(6,'Dark',["NA","NA",7.5,0])
widetypes1.insert(16, "Steel", ['NA','NA',17,0])
widetypes1.insert(16, "Flying", [1.626316,61.247368,18,19])
widetypes1.loc['Index'] = list((range(1,19)))
widetypes1
```

[319]:
```
Type1            Bug     Dragon   Electric  Fairy   Fighting       Fire Dark  \
Height      0.900000   2.666667   0.855556   0.95   1.185714   1.216667   NA
Weight     22.991667  76.600000  31.788889  23.75  54.285714  48.025000   NA
Index       1.000000   2.000000   3.000000   4.00   5.000000   6.000000    7
Frequency  12.000000   3.000000   9.000000   2.00   7.000000  12.000000    0

Type1          Ghost      Grass   Ground    Ice     Normal     Poison  \
Height      1.466667   1.083333   0.8500   1.55   0.986364   1.221429
Weight     13.566667  27.991667  45.2625  48.00  50.086364  27.314286
Index       8.000000   9.000000  10.0000  11.00  12.000000  13.000000
Frequency   3.000000  12.000000   8.0000   2.00  22.000000  14.000000

Type1       Psychic       Rock      Water     Flying Steel
Height       1.2500   1.844444   1.300000   1.626316    NA
Weight      51.5625  87.611111  57.967857  61.247368    NA
Index       14.0000  15.000000  16.000000  17.000000    18
Frequency    8.0000   9.000000  28.000000  19.000000     0
```

[231]:
```
print(pd.
  ↪pivot_table(gen1,values=['Height','Weight'],columns='Type2',aggfunc='mean'))
chuckit = gen1.groupby('Type2')
print(chuckit['Name'].count())
```

```
Type2       Fairy  Fighting     Flying  Grass      Ground    Ice     Poison  \
```

```
Height   0.933333       1.3   1.626316   0.65    2.383333    1.9   1.100000
Weight  24.000000      54.0  61.247368  17.45  126.166667  157.5  21.305263

Type2     Psychic    Rock  Steel   Water
Height   1.283333    1.45   0.65   0.800
Weight  59.600000  117.50  33.00  23.625
Type2
Fairy        3
Fighting     1
Flying      19
Grass        2
Ground       6
Ice          3
Poison      19
Psychic      6
Rock         2
Steel        2
Water        4
Name: Name, dtype: int64
```

[229]:
```
widetypes2 = pd.
 ↪pivot_table(gen1,values=['Height','Weight'],columns='Type2',aggfunc='mean')
widetypes2.loc['Frequency'] = chuckit['Name'].count()
widetypes2
```

[229]:
```
Type2          Fairy  Fighting     Flying  Grass      Ground    Ice  \
Height      0.933333       1.3   1.626316   0.65    2.383333    1.9
Weight     24.000000      54.0  61.247368  17.45  126.166667  157.5
Frequency   3.000000       1.0  19.000000   2.00    6.000000    3.0

Type2         Poison    Psychic    Rock  Steel   Water
Height      1.100000   1.283333    1.45   0.65   0.800
Weight     21.305263  59.600000  117.50  33.00  23.625
Frequency  19.000000   6.000000    2.00   2.00   4.000
```

1.11 [8] Data Cleaning

i) removing columns (just above we did inserting)

[249]:
```
gen1.drop(columns=['Type1','Type2','Height','Weight']).set_index('DexNo')
```

[249]:
```
              Name  Legendary  Mythical
DexNo
1        Bulbasaur          0         0
2          Ivysaur          0         0
3         Venusaur          0         0
4       Charmander          0         0
5       Charmeleon          0         0
```

```
…           …        …       …
147       Dratini          0       0
148     Dragonair          0       0
149     Dragonite          0       0
150        Mewtwo          1       0
151           Mew          0       1

[151 rows x 3 columns]
```

ii) removing/altering missing value rows

[250]:
```
gen1.dropna(subset=['Type2'])
```

[250]:

	DexNo	Name	Type1	Type2	Height	Weight	Legendary	Mythical
0	1	Bulbasaur	Grass	Poison	0.7	6.9	0	0
1	2	Ivysaur	Grass	Poison	1.0	13.0	0	0
2	3	Venusaur	Grass	Poison	2.0	100.0	0	0
5	6	Charizard	Fire	Flying	1.7	90.5	0	0
11	12	Butterfree	Bug	Flying	1.1	32.0	0	0
..	…	…	…	…	…	…	…	
141	142	Aerodactyl	Rock	Flying	1.8	59.0	0	0
143	144	Articuno	Ice	Flying	1.7	55.4	1	0
144	145	Zapdos	Electric	Flying	1.6	52.6	1	0
145	146	Moltres	Fire	Flying	2.0	60.0	1	0
148	149	Dragonite	Dragon	Flying	2.2	210.0	0	0

[67 rows x 8 columns]

[311]:
```
gen1dupe = gen1.copy()
for i in range(151): #loops through and doubles up the typing
    if str(gen1dupe.iloc[i,3]) == 'nan':
        gen1dupe.iloc[i,3] = gen1dupe.iloc[i,2]
print(gen1dupe)
gen1dupe['Legendary'] = gen1dupe['Legendary'].astype(bool)
gen1dupe['Mythical'] = gen1dupe['Mythical'].astype(bool)

if str(gen1dupe.iloc[4,3]) == 'nan': #pointless checker (for fun)
    print('f')
else:
    print(gen1dupe.iloc[4,3])
    print(len(str(gen1dupe.iloc[4,3])))
```

	DexNo	Name	Type1	Type2	Height	Weight	Legendary	Mythical
0	1	Bulbasaur	Grass	Poison	0.7	6.9	0	0
1	2	Ivysaur	Grass	Poison	1.0	13.0	0	0
2	3	Venusaur	Grass	Poison	2.0	100.0	0	0
3	4	Charmander	Fire	Fire	0.6	8.5	0	0
4	5	Charmeleon	Fire	Fire	1.1	19.0	0	0

```
..     …          …       …       …      …      …         …        …
146    147     Dratini   Dragon   Dragon     1.8     3.3          0         0
147    148   Dragonair   Dragon   Dragon     4.0    16.5          0         0
148    149   Dragonite   Dragon   Flying     2.2   210.0          0         0
149    150      Mewtwo  Psychic  Psychic     2.0   122.0          1         0
150    151         Mew  Psychic  Psychic     0.4     4.0          0         1

[151 rows x 8 columns]
Fire
4
```

[286]:
```
print(gen1dupe.set_index('DexNo').to_string())
```

```
             Name     Type1     Type2  Height  Weight  Legendary  Mythical
DexNo
1       Bulbasaur     Grass    Poison     0.7     6.9      False     False
2         Ivysaur     Grass    Poison     1.0    13.0      False     False
3        Venusaur     Grass    Poison     2.0   100.0      False     False
4      Charmander      Fire      Fire     0.6     8.5      False     False
5      Charmeleon      Fire      Fire     1.1    19.0      False     False
6       Charizard      Fire    Flying     1.7    90.5      False     False
7        Squirtle     Water     Water     0.5     9.0      False     False
8       Wartortle     Water     Water     1.0    22.5      False     False
9       Blastoise     Water     Water     1.6    85.5      False     False
10       Caterpie       Bug       Bug     0.3     2.9      False     False
11         Metapod       Bug       Bug     0.7     9.9      False     False
12     Butterfree       Bug    Flying     1.1    32.0      False     False
13         Weedle       Bug    Poison     0.3     3.2      False     False
14         Kakuna       Bug    Poison     0.6    10.0      False     False
15       Beedrill       Bug    Poison     1.0    29.5      False     False
16         Pidgey    Normal    Flying     0.3     1.8      False     False
17      Pidgeotto    Normal    Flying     1.1    30.0      False     False
18        Pidgeot    Normal    Flying     1.5    39.5      False     False
19        Rattata    Normal    Normal     0.3     3.5      False     False
20       Raticate    Normal    Normal     0.7    18.5      False     False
21        Spearow    Normal    Flying     0.3     2.0      False     False
22         Fearow    Normal    Flying     1.2    38.0      False     False
23          Ekans    Poison    Poison     2.0     6.9      False     False
24          Arbok    Poison    Poison     3.5    65.0      False     False
25        Pikachu  Electric  Electric     0.4     6.0      False     False
26         Raichu  Electric  Electric     0.8    30.0      False     False
27      Sandshrew    Ground    Ground     0.6    12.0      False     False
28      Sandslash    Ground    Ground     1.0    29.5      False     False
29       Nidoran    Poison    Poison     0.4     7.0      False     False
30       Nidorina    Poison    Poison     0.8    20.0      False     False
31      Nidoqueen    Poison    Ground     1.3    60.0      False     False
32       Nidoran    Poison    Poison     0.5     9.0      False     False
33       Nidorino    Poison    Poison     0.9    19.5      False     False
```

34	Nidoking	Poison	Ground	1.4	62.0	False	False
35	Clefairy	Fairy	Fairy	0.6	7.5	False	False
36	Clefable	Fairy	Fairy	1.3	40.0	False	False
37	Vulpix	Fire	Fire	0.6	9.9	False	False
38	Ninetales	Fire	Fire	1.1	19.9	False	False
39	Jigglypuff	Normal	Fairy	0.5	5.5	False	False
40	Wigglytuff	Normal	Fairy	1.0	12.0	False	False
41	Zubat	Poison	Flying	0.8	7.5	False	False
42	Golbat	Poison	Flying	1.6	55.0	False	False
43	Oddish	Grass	Poison	0.5	5.4	False	False
44	Gloom	Grass	Poison	0.8	8.6	False	False
45	Vileplume	Grass	Poison	1.2	18.6	False	False
46	Paras	Bug	Grass	0.3	5.4	False	False
47	Parasect	Bug	Grass	1.0	29.5	False	False
48	Venonat	Bug	Poison	1.0	30.0	False	False
49	Venomoth	Bug	Poison	1.5	12.5	False	False
50	Diglett	Ground	Ground	0.2	0.8	False	False
51	Dugtrio	Ground	Ground	0.7	33.3	False	False
52	Meowth	Normal	Normal	0.4	4.2	False	False
53	Persian	Normal	Normal	1.0	32.0	False	False
54	Psyduck	Water	Water	0.8	19.6	False	False
55	Golduck	Water	Water	1.7	76.6	False	False
56	Mankey	Fighting	Fighting	0.5	28.0	False	False
57	Primeape	Fighting	Fighting	1.0	32.0	False	False
58	Growlithe	Fire	Fire	0.7	19.0	False	False
59	Arcanine	Fire	Fire	1.9	155.0	False	False
60	Poliwag	Water	Water	0.6	12.4	False	False
61	Poliwhirl	Water	Water	1.0	20.0	False	False
62	Poliwrath	Water	Fighting	1.3	54.0	False	False
63	Abra	Psychic	Psychic	0.9	19.5	False	False
64	Kadabra	Psychic	Psychic	1.3	56.5	False	False
65	Alakazam	Psychic	Psychic	1.5	48.0	False	False
66	Machop	Fighting	Fighting	0.8	19.5	False	False
67	Machoke	Fighting	Fighting	1.5	70.5	False	False
68	Machamp	Fighting	Fighting	1.6	130.0	False	False
69	Bellsprout	Grass	Poison	0.7	4.0	False	False
70	Weepinbell	Grass	Poison	1.0	6.4	False	False
71	Victreebel	Grass	Poison	1.7	15.5	False	False
72	Tentacool	Water	Poison	0.9	45.5	False	False
73	Tentacruel	Water	Poison	1.6	55.0	False	False
74	Geodude	Rock	Ground	0.4	20.0	False	False
75	Graveler	Rock	Ground	1.0	105.0	False	False
76	Golem	Rock	Ground	1.4	300.0	False	False
77	Ponyta	Fire	Fire	1.0	30.0	False	False
78	Rapidash	Fire	Fire	1.7	95.0	False	False
79	Slowpoke	Water	Psychic	1.2	36.0	False	False
80	Slowbro	Water	Psychic	1.6	78.5	False	False
81	Magnemite	Electric	Steel	0.3	6.0	False	False

82	Magneton	Electric	Steel	1.0	60.0	False	False
83	Farfetch'd	Normal	Flying	0.8	15.0	False	False
84	Doduo	Normal	Flying	1.4	39.2	False	False
85	Dodrio	Normal	Flying	1.8	85.2	False	False
86	Seel	Water	Water	1.1	90.0	False	False
87	Dewgong	Water	Ice	1.7	120.0	False	False
88	Grimer	Poison	Poison	0.9	30.0	False	False
89	Muk	Poison	Poison	1.2	30.0	False	False
90	Shellder	Water	Water	0.3	4.0	False	False
91	Cloyster	Water	Ice	1.5	132.5	False	False
92	Gastly	Ghost	Poison	1.3	0.1	False	False
93	Haunter	Ghost	Poison	1.6	0.1	False	False
94	Gengar	Ghost	Poison	1.5	40.5	False	False
95	Onix	Rock	Ground	8.8	210.0	False	False
96	Drowzee	Psychic	Psychic	1.0	32.4	False	False
97	Hypno	Psychic	Psychic	1.6	75.6	False	False
98	Krabby	Water	Water	0.4	6.5	False	False
99	Kingler	Water	Water	1.3	60.0	False	False
100	Voltorb	Electric	Electric	0.5	10.4	False	False
101	Electrode	Electric	Electric	1.2	66.6	False	False
102	Exeggcute	Grass	Psychic	0.4	2.5	False	False
103	Exeggutor	Grass	Psychic	2.0	120.0	False	False
104	Cubone	Ground	Ground	0.4	6.5	False	False
105	Marowak	Ground	Ground	1.0	45.0	False	False
106	Hitmonlee	Fighting	Fighting	1.5	49.8	False	False
107	Hitmonchan	Fighting	Fighting	1.4	50.2	False	False
108	Lickitung	Normal	Normal	1.2	65.5	False	False
109	Koffing	Poison	Poison	0.6	1.0	False	False
110	Weezing	Poison	Poison	1.2	9.5	False	False
111	Rhyhorn	Ground	Rock	1.0	115.0	False	False
112	Rhydon	Ground	Rock	1.9	120.0	False	False
113	Chansey	Normal	Normal	1.1	34.6	False	False
114	Tangela	Grass	Grass	1.0	35.0	False	False
115	Kangaskhan	Normal	Normal	2.2	80.0	False	False
116	Horsea	Water	Water	0.4	8.0	False	False
117	Seadra	Water	Water	1.2	25.0	False	False
118	Goldeen	Water	Water	0.6	15.0	False	False
119	Seaking	Water	Water	1.3	39.0	False	False
120	Staryu	Water	Water	0.8	34.5	False	False
121	Starmie	Water	Psychic	1.1	80.0	False	False
122	Mr. Mime	Psychic	Fairy	1.3	54.5	False	False
123	Scyther	Bug	Flying	1.5	56.0	False	False
124	Jynx	Ice	Psychic	1.4	40.6	False	False
125	Electabuzz	Electric	Electric	1.1	30.0	False	False
126	Magmar	Fire	Fire	1.3	44.5	False	False
127	Pinsir	Bug	Bug	1.5	55.0	False	False
128	Tauros	Normal	Normal	1.4	88.4	False	False
129	Magikarp	Water	Water	0.9	10.0	False	False

```
130      Gyarados     Water    Flying     6.5   235.0      False     False
131        Lapras     Water       Ice     2.5   220.0      False     False
132         Ditto    Normal    Normal     0.3     4.0      False     False
133         Eevee    Normal    Normal     0.3     6.5      False     False
134      Vaporeon     Water     Water     1.0    29.0      False     False
135       Jolteon  Electric  Electric     0.8    24.5      False     False
136       Flareon      Fire      Fire     0.9    25.0      False     False
137       Porygon    Normal    Normal     0.8    36.5      False     False
138       Omanyte      Rock     Water     0.4     7.5      False     False
139       Omastar      Rock     Water     1.0    35.0      False     False
140        Kabuto      Rock     Water     0.5    11.5      False     False
141      Kabutops      Rock     Water     1.3    40.5      False     False
142    Aerodactyl      Rock    Flying     1.8    59.0      False     False
143       Snorlax    Normal    Normal     2.1   460.0      False     False
144      Articuno       Ice    Flying     1.7    55.4       True     False
145        Zapdos  Electric    Flying     1.6    52.6       True     False
146       Moltres      Fire    Flying     2.0    60.0       True     False
147       Dratini    Dragon    Dragon     1.8     3.3      False     False
148     Dragonair    Dragon    Dragon     4.0    16.5      False     False
149     Dragonite    Dragon    Flying     2.2   210.0      False     False
150        Mewtwo   Psychic   Psychic     2.0   122.0       True     False
151           Mew   Psychic   Psychic     0.4     4.0      False      True
```

iii) filling missing value rows

[295]:
```
print(gen1.fillna(0).set_index('DexNo')) #first dataframe, just filled with␣
 ↪Zeros or N/As
print(gen1.fillna({"Type2": "None"}).set_index('DexNo')) #sae as before but␣
 ↪only targets type2 column
```

```
             Name    Type1   Type2  Height  Weight  Legendary  Mythical
DexNo
1       Bulbasaur    Grass  Poison     0.7     6.9          0         0
2         Ivysaur    Grass  Poison     1.0    13.0          0         0
3        Venusaur    Grass  Poison     2.0   100.0          0         0
4      Charmander     Fire       0     0.6     8.5          0         0
5      Charmeleon     Fire       0     1.1    19.0          0         0
…           …       …      …      …      …         …       …
147       Dratini   Dragon       0     1.8     3.3          0         0
148     Dragonair   Dragon       0     4.0    16.5          0         0
149     Dragonite   Dragon  Flying     2.2   210.0          0         0
150        Mewtwo  Psychic       0     2.0   122.0          1         0
151           Mew  Psychic       0     0.4     4.0          0         1

[151 rows x 7 columns]
             Name    Type1   Type2  Height  Weight  Legendary  Mythical
DexNo
1       Bulbasaur    Grass  Poison     0.7     6.9          0         0
```

```
2         Ivysaur    Grass  Poison     1.0    13.0          0          0
3        Venusaur    Grass  Poison     2.0   100.0          0          0
4      Charmander     Fire    None     0.6     8.5          0          0
5      Charmeleon     Fire    None     1.1    19.0          0          0
…             …      …       …        …       …          …
147       Dratini   Dragon    None     1.8     3.3          0          0
148     Dragonair   Dragon    None     4.0    16.5          0          0
149     Dragonite   Dragon  Flying     2.2   210.0          0          0
150        Mewtwo  Psychic    None     2.0   122.0          1          0
151           Mew  Psychic    None     0.4     4.0          0          1

[151 rows x 7 columns]
```

iv) fixing or replacing inconsistent values case study: with the introduction of the fairy typing, several pokemon in XY/LGPE (gen6&7) were given updated 'fairy' typing instead of 'normal' or single typed for various pokemon - let's revert this!

[23]:
```
gen1[(gen1['Type1'] == 'Fairy') | (gen1['Type2'] == 'Fairy')]
gen1.iloc[[33,34,35,36,37,38,39,89,100,120,121,122]]
```

[23]:

	DexNo	Name	Type1	Type2	Height	Weight	Legendary	Mythical
33	34	Nidoking	Poison	Ground	1.4	62.0	0	0
34	35	Clefairy	Fairy	NaN	0.6	7.5	0	0
35	36	Clefable	Fairy	NaN	1.3	40.0	0	0
36	37	Vulpix	Fire	NaN	0.6	9.9	0	0
37	38	Ninetales	Fire	NaN	1.1	19.9	0	0
38	39	Jigglypuff	Normal	Fairy	0.5	5.5	0	0
39	40	Wigglytuff	Normal	Fairy	1.0	12.0	0	0
89	90	Shellder	Water	NaN	0.3	4.0	0	0
100	101	Electrode	Electric	NaN	1.2	66.6	0	0
120	121	Starmie	Water	Psychic	1.1	80.0	0	0
121	122	Mr. Mime	Psychic	Fairy	1.3	54.5	0	0
122	123	Scyther	Bug	Flying	1.5	56.0	0	0

[312]:
```
gen1dupe['Type1'] = gen1dupe['Type1'].replace({"Fairy":"Normal","Normal":
  ↪"NORMAL"})
gen1dupe['Type2'] = gen1dupe['Type2'].replace({"Fairy":""})
gen1dupe.iloc[[33,34,35,36,37,38,39,89,100,120,121,122]]
```

[312]:

	DexNo	Name	Type1	Type2	Height	Weight	Legendary	\
33	34	Nidoking	Poison	Ground	1.4	62.0	False	
34	35	Clefairy	Normal		0.6	7.5	False	
35	36	Clefable	Normal		1.3	40.0	False	
36	37	Vulpix	Fire	Fire	0.6	9.9	False	
37	38	Ninetales	Fire	Fire	1.1	19.9	False	
38	39	Jigglypuff	NORMAL		0.5	5.5	False	
39	40	Wigglytuff	NORMAL		1.0	12.0	False	
89	90	Shellder	Water	Water	0.3	4.0	False	

```
100    101   Electrode  Electric  Electric     1.2    66.6      False
120    121     Starmie     Water   Psychic     1.1    80.0      False
121    122    Mr. Mime   Psychic               1.3    54.5      False
122    123     Scyther       Bug    Flying     1.5    56.0      False

     Mythical
33      False
34      False
35      False
36      False
37      False
38      False
39      False
89      False
100     False
120     False
121     False
122     False
```

v) standardising text, removing duplicates

[315]:
```
gen1['Name'].str.lower()
```

[315]:
```
0       bulbasaur
1         ivysaur
2        venusaur
3      charmander
4      charmeleon
           …
146       dratini
147     dragonair
148     dragonite
149        mewtwo
150           mew
Name: Name, Length: 151, dtype: object
```

[316]:
```
gen1 = gen1.drop_duplicates() #this would remove duplicates if there were any...
```

vi) changing data typeremember the wide pivot table? the index row was messed up….

[321]:
```
widetypes1
```

[321]:
```
Type1            Bug     Dragon   Electric  Fairy   Fighting       Fire Dark  \
Height      0.900000   2.666667   0.855556   0.95   1.185714   1.216667   NA
Weight     22.991667  76.600000  31.788889  23.75  54.285714  48.025000   NA
Index       1.000000   2.000000   3.000000   4.00   5.000000   6.000000    7
Frequency  12.000000   3.000000   9.000000   2.00   7.000000  12.000000    0
```

```
Type1          Ghost      Grass   Ground    Ice     Normal     Poison  \
Height      1.466667   1.083333   0.8500   1.55   0.986364   1.221429
Weight     13.566667  27.991667  45.2625  48.00  50.086364  27.314286
Index       8.000000   9.000000  10.0000  11.00  12.000000  13.000000
Frequency   3.000000  12.000000   8.0000   2.00  22.000000  14.000000

Type1       Psychic       Rock      Water     Flying Steel
Height       1.2500   1.844444   1.300000   1.626316    NA
Weight      51.5625  87.611111  57.967857  61.247368    NA
Index       14.0000  15.000000  16.000000  17.000000    18
Frequency    8.0000   9.000000  28.000000  19.000000     0
```

[327]:
```
widetypes1.loc['Index'] = widetypes1.loc['Index'].astype(int)
print(widetypes1.loc['Index'])
```

```
Type1
Bug          1.0
Dragon       2.0
Electric     3.0
Fairy        4.0
Fighting     5.0
Fire         6.0
Dark           7
Ghost        8.0
Grass        9.0
Ground      10.0
Ice         11.0
Normal      12.0
Poison      13.0
Psychic     14.0
Rock        15.0
Water       16.0
Flying      17.0
Steel         18
Name: Index, dtype: object
```

seems not to work.. let's try something else

[350]:
```
print(widetypes1.iloc[[2],[1]]) #appearing as a float...
print()
print(widetypes1.iloc[3,5],widetypes1.iloc[3,5].dtype)
print(widetypes1.iloc[3,5].astype(int)) #we can change them

for i in range(len(widetypes1.loc['Index'])):
    widetypes1.iloc[2,i] += 0.001
    widetypes1.iloc[2,i] = int(round(widetypes1.iloc[2,i]))

print(widetypes1.iloc[3,5],widetypes1.iloc[3,5].dtype)
```

```
widetypes1
```

```
Type1  Dragon
Index     2.0

12.0 float64
12
12.0 float64
```

[350]:
Type1	Bug	Dragon	Electric	Fairy	Fighting	Fire	Dark	\
Height	0.900000	2.666667	0.855556	0.95	1.185714	1.216667	NA	
Weight	22.991667	76.600000	31.788889	23.75	54.285714	48.025000	NA	
Index	1.000000	2.000000	3.000000	4.00	5.000000	6.000000	7	
Frequency	12.000000	3.000000	9.000000	2.00	7.000000	12.000000	0	

Type1	Ghost	Grass	Ground	Ice	Normal	Poison	\
Height	1.466667	1.083333	0.8500	1.55	0.986364	1.221429	
Weight	13.566667	27.991667	45.2625	48.00	50.086364	27.314286	
Index	8.000000	9.000000	10.0000	11.00	12.000000	13.000000	
Frequency	3.000000	12.000000	8.0000	2.00	22.000000	14.000000	

Type1	Psychic	Rock	Water	Flying	Steel
Height	1.2500	1.844444	1.300000	1.626316	NA
Weight	51.5625	87.611111	57.967857	61.247368	NA
Index	14.0000	15.000000	16.000000	17.000000	18
Frequency	8.0000	9.000000	28.000000	19.000000	0

[353]:
```
#clear fail, just don't do this (avoid) :))
```